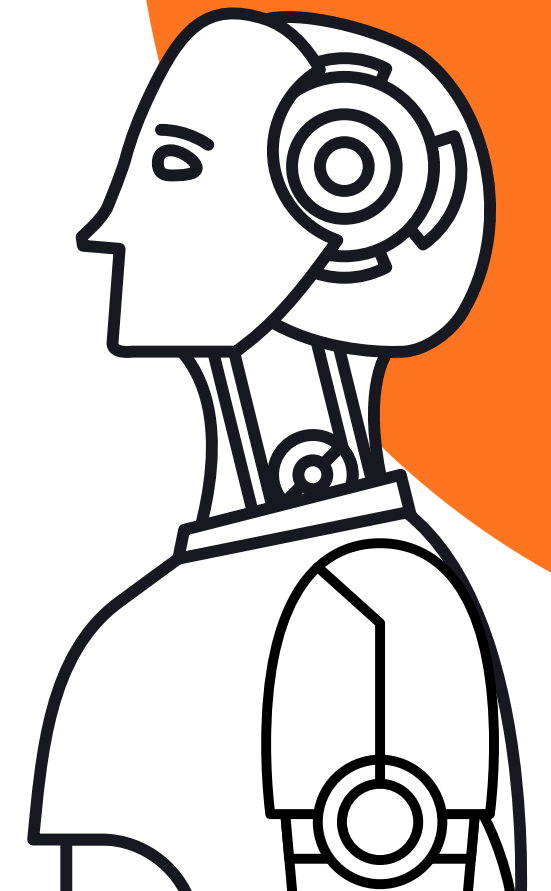
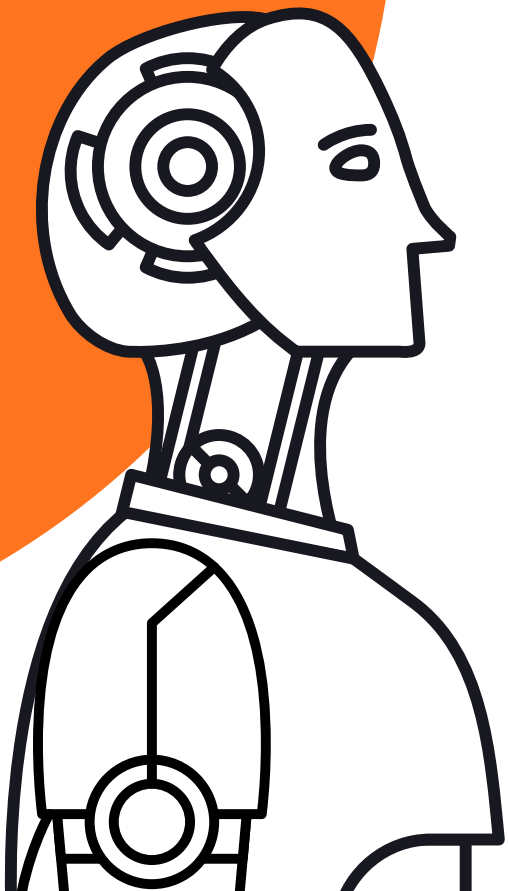


POSTMAN

POSTMAN

THE API PLATFORM FOR
DEVELOPERS

Jeann Victor, Nicolas Rodrigues, Pedro Augusto e Thiago Martins



CONTEXT AND HISTORY

- Created in 2012
- Started as a Chrome extension
- Developed by Abhinav Asthana
- Became one of the most popular API platforms
- Used by over 30 million developers worldwide
- Why was Postman created?



WHY POSTMAN BECAME POPULAR?

Easy to Learn

- Simple and intuitive interface
- Beginner-friendly design
- No advanced setup required

Faster API Testing

- Send requests in seconds
- View responses instantly
- Supports REST, GraphQL, and SOAP APIs

Multiple HTTP Methods:

- GET
- POST
- PUT
- DELETE,
- PATCH

Team Collaboration

- Share collections and environments
- Workspaces for teams
- Cloud synchronization

Documentation

- Generate API documentation automatically
- Easier communication between frontend and backend teams

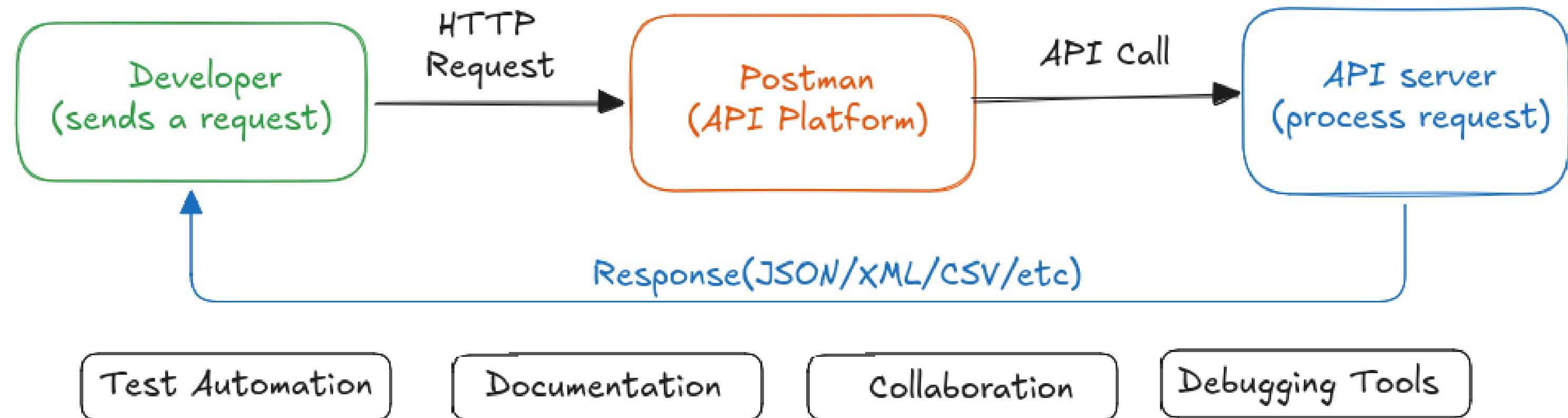
Automation & Debugging

- Automated API tests
- Environment variables
- Request history and debugging tools



WHAT IS POSTMAN?

"Postman is an API platform that helps developers build, test, manage, and document APIs efficiently."



COLLECTIONS E ENVIRONMENTS

PAGE 05

Collections — group and organize requests by project, share with team, auto-generate documentation.

Environments — store variables like `{{base_url}}` and `{{token}}`, switch between dev/staging/production instantly.

The screenshot shows the Postman application interface. On the left sidebar, the 'Environments' section is active, listing several environments: 'batsignal', 'batsignal', 'batsignalButton', 'cli-test', 'Covid VS Stock Market', 'discoursebot', 'GitHub GraphQL', 'GitHub-Slack', 'hue-proxy', and 'links'. The 'discoursebot' environment is highlighted. A tooltip with a lightbulb icon and the text 'Set inactive' is positioned over the 'discoursebot' entry. The main panel displays the 'Overview' tab for the 'discoursebot' environment. It features a table with the following columns: 'VARIABLE', 'INITIAL VALUE', and 'CURRENT VALUE'. The table contains four rows of variables, each with a checked checkbox in the first column:

	VARIABLE	INITIAL VALUE	CURRENT VALUE
☒	baseUrl	https://community.postman.com	https://community.postman.com
☒	id	10	10
☒	topics		
☒	topic_url		

Below the table is a link to 'Add a new variable'. To the right of the table, there are buttons for 'Save', 'Share', and 'Persist All', along with a 'Reset All' button. The top of the interface shows navigation tabs like 'Home', 'Workspaces', 'Reports', and 'Explore', along with a search bar and user profile icons.

WHAT IS AN API? THE INVISIBLE BRIDGE

"A set of code-based instructions that enable different software components to COMMUNICATE and SHARE DATA."

PAGE 06

REAL WORLD ANALOGY:

THE WAITER:

- YOU (CLIENT) DON'T GO TO THE KITCHEN.
- THE WAITER TAKES YOUR ORDER (REQUEST)
- THE WAITER BRINGS YOUR FOOD (RESPONSE)
- THE API IS THE WAITER

TECHNICAL DEFINITION: API = APPLICATION PROGRAMMING INTERFACE

- BRIDGE BETWEEN SOFTWARE COMPONENTS
- DEFINES HOW THEY CAN INTERACT
- USES PROTOCOLS LIKE HTTP
- RETURNS STRUCTURED DATA (JSON, XML)

APIS LET DIFFERENT PROGRAMS TALK TO EACH OTHER WITHOUT NEEDING TO KNOW HOW EACH ONE WORKS INTERNALLY.

CLIENT-SERVER ARCHITECTURE

PAGE 07

"A model where clients request resources and servers provide them. They communicate only via requests and responses."

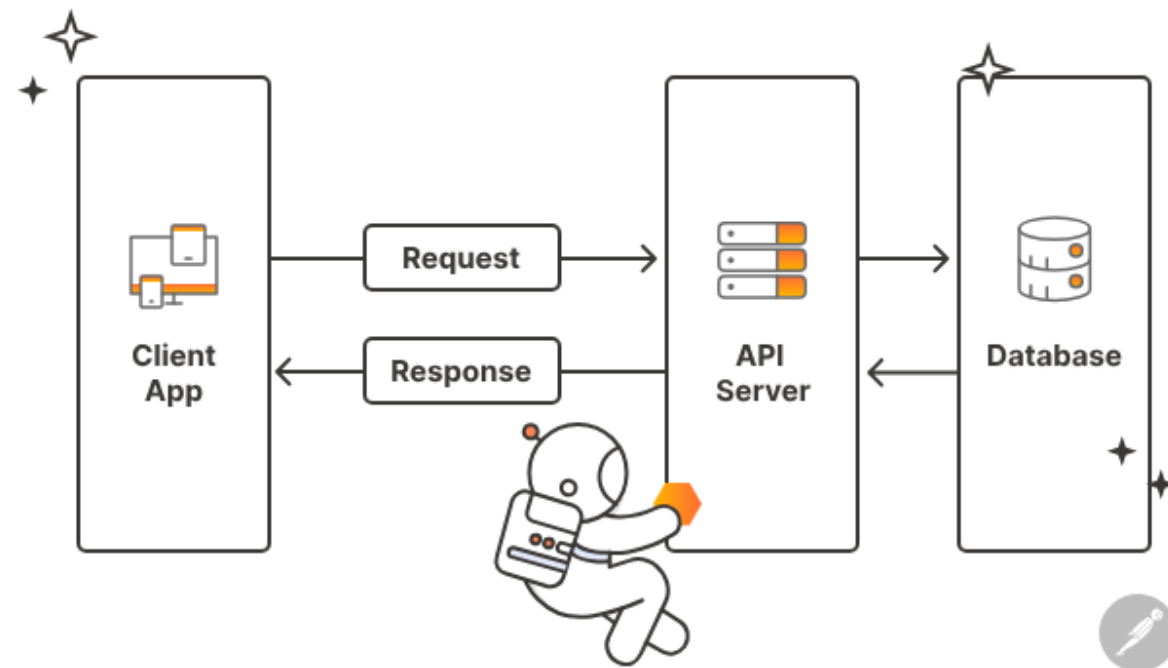
KEY CHARACTERISTICS:

CLIENT:

- INITIATES COMMUNICATION
- SENDS REQUESTS
- DISPLAYS RESULTS
- NO DIRECT DATABASE ACCESS
- **EXAMPLE:** CLIENT APP

SERVER:

- WAITS FOR REQUESTS
- PROCESSES REQUESTS
- SENDS RESPONSES
- MANAGES DATA & BUSINESS LOGIC
- **EXAMPLE:** API SERVER



POSTMAN ACTS AS A CLIENT. IT SENDS HTTP REQUESTS TO SERVERS AND SHOWS YOU THE RESPONSES. NO SERVER CODE NEEDED.

HTTP (HYPERTEXT TRANSFER PROTOCOL)

PAGE 08

“ A set of rules that define how data is transferred between a client and server on the internet .”

WHAT IS INCLUDED IN AN HTTP REQUEST?

METHODS:

- **GET:** FETCH A RESOURCE
- **POST:** ADD NEW DATA
- **PUT:** UPDATE EXISTING DATA
- **DELETE:** REMOVE A RESOURCE

HTTP MESSAGE STRUCTURE:

1. METHOD + URL + VERSION
 2. HEADERS (METADATA)
 3. BODY (OPTIONAL DATA)
- **URL:** SCHEME + DOMAIN + PATH
 - **HEADERS:** HOST, USER-AGENT, ACCEPT, CONTENT-TYPE
 - **BODY:** JSON, XML, OR FORM DATA

REAL HTTP REQUEST EXAMPLE:

```
POST /API/RESOURCE HTTP/1.1
HOST: EXAMPLE.COM
USER-AGENT: MOZILLA/5.0
CONTENT-TYPE: APPLICATION/JSON
CONTENT-LENGTH: 35
{
  "KEY1": "VALUE1",
  "KEY2": "VALUE2"
}
```

Each request is independent. The server doesn't remember previous requests.

HTTP (HYPERTEXT TRANSFER PROTOCOL)

"After processing the request, the server sends a response with the requested data and metadata"

PAGE 09

WHAT IS INCLUDED IN AN HTTP RESPONSE?

3 COMPONENTS OF AN HTTP RESPONSE:

- **HTTP STATUS CODE:** 3-DIGIT NUMBER INDICATING THE RESULT (E.G., 200, 404, 500)
- **RESPONSE HEADERS:** METADATA ABOUT THE RESPONSE (CONTENT-TYPE, CONTENT-LENGTH, SERVER)
- **BODY (OPTIONAL):** THE ACTUAL DATA REQUESTED (JSON, HTML, IMAGE, VIDEO)

REAL HTTP RESPONSE EXAMPLE:

```
HTTP/1.1 200 OK
CONTENT-TYPE: APPLICATION/JSON
CONTENT-LENGTH: 156
SERVER: NGINX/1.18.0
CACHE-CONTROL: NO-CACHE
{
  "ID": 1,
  "NAME": "JOÃO SILVA",
  "EMAIL": "JOAO@EMAIL.COM"
}
```

200 = Good | 400s = You messed up (Client) | 500s = Server messed up

REST APIS: ORGANIZING THE WEB WITH STYLE

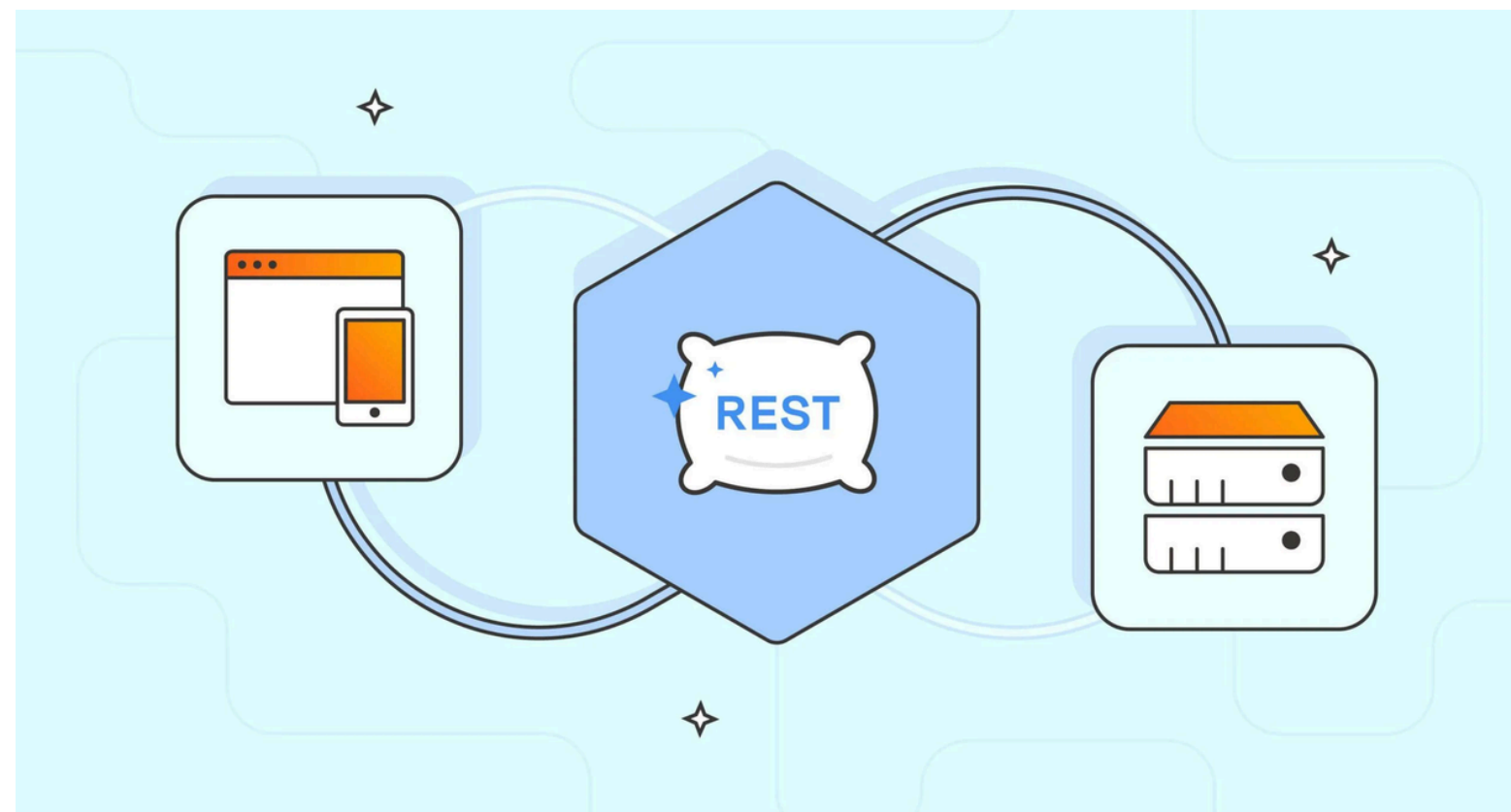
"An architectural style, not a protocol. It uses HTTP and standard web protocols for simple, uniform communication."

KEY CONSTRAINTS:

- UNIFORM INTERFACE
- CLIENT-SERVER SEPARATION
- STATELESS OPERATIONS
- CACHEABLE RESPONSES
- LAYERED SYSTEM

HTTP METHODS IN REST AND EXAMPLES:

- **GET:** /API/PRODUCTS
- **POST:** /API/ORDERS
- **PUT:** /API/USERS/123
- **PATCH:** /API/USERS/123
- **DELETE:** /API/ORDERS/456



TECHNICAL GLOSSARY

- API (application programming interface)
- HTTP (Hypertext transfer protocol)
- Endpoint
- JSON
- Client-server Architecture
- Status Code
- GET Method
- POST Method
- PUT Method
- DELETE Method
- PATCH Method

LET'S PRACTICE!!!

LIVE API CALLS WITH POSTMAN

Testing real HTTP requests in public APIs

How this works?

- Open web.postman.co in your browser
- Sign in with Google or Github
- Create a new Workspace
- Click New Collection → New HTTP Request

Follow along — try it yourself!



LET'S PRACTICE!!!

LIVE API CALLS WITH POSTMAN

Testing real HTTP requests in public APIs

The APIs

▪ Simple Books API

<https://simple-books-api.click>

A simple bookstore REST API.

Supports authentication, listing books and creating new orders.

Free, public and regularly maintained.

▪ PokeAPI

<https://pokeapi.co/api/v2/pokemon/>

The largest Pokémon RESTful API.

Read-only — no authentication needed.

Returns stats, types, abilities & sprites.

Free, public and regularly maintained.

Both APIs are free, open to use and regularly maintained.



API #1: Simple-books

Endpoints

Authenticate as a client, browse available books, and place a new order.

```
POST https://simple-books-api.click/api-clients/
```

→ Registers the API client | 201 Created

```
GET https://simple-books-api.click/books
```

→ Lists all available books | 200 OK

```
POST https://simple-books-api.click/orders
```

→ Creates a new order | 201 Created

POST /api-clients/ – Body

```
{  
  "clientName": "Your Name",  
  "clientEmail": "you@example.com"  
}
```

POST /orders – Body

```
{  
  "bookId":  
  "customerName": "Your Name"  
}
```

- Pay attention to the status codes returned after each request!



API #2: PokéAPI

Endpoints

The largest Pokémon RESTful API. Read-only. No authentication required.

```
GET https://pokeapi.co/api/v2/pokemon/pikachu
→ Returns all info about Pikachu | 200 OK

GET https://pokeapi.co/api/v2/pokemon/charizard
→ Returns all info about Charizard | 200 OK

GET https://pokeapi.co/api/v2/pokemon/[your-choice]
→ Replace with any Pokémon name!
```

Explore the Response

`name` → The Pokémon name

`types[].type.name` → Type(s): fire, water, electric...

`abilities[].ability.name` → List of abilities

`sprites.front_default` → URL of the Pokémon image → open in browser!

`stats[].base_stat` → HP, Attack, Defense values



What did we see? - Status Codes

Recap

Code	Meaning	
200 OK	Success — data returned	PokeAPI GET, Simple Books GET /books
201 Created	Resource created	Simple Books /api-clients & /orders
204 No Content	Success, nothing to return	Some DELETE operations
401 Unauthorized	Missing or invalid token	Simple Books /orders without token
404 Not Found	Resource does not exist	Wrong Pokémon name or invalid ID

APIs are everywhere — Spotify, iFood, Instagram all work exactly like this.
Postman lets you see what happens under the hood — without writing a single line of code.



REFERENCES

Postman Docs

Learn how to use Postman. Search the docs and support resources!

